



AES and SM4

Step-by-Step Encryption, Differential Fault Analysis, and Key Recovery

Educational Technical Notes for PFE 5A

Student: Guilherme Specht

Supervisor: Prof. Philippe Maurine

Institution: Polytech Montpellier

Industrial context: STMicroelectronics

Date: 24 October 2026

Contents

1 Purpose and Notation **3**

2 AES-128: Encryption from the Beginning **4**
2.1 Block, key, and round structure . . . 4
2.2 Initial AddRoundKey . . . 4
2.3 SubBytes . . . 4
2.4 ShiftRows . . . 5
2.5 MixColumns . . . 5
2.6 Round keys . . . 5

3 Piret–Quisquater Differential Fault Analysis of AES **6**
3.1 Where the fault is injected . . . 6
3.2 Propagation through MixColumns . . . 6
3.3 Which ciphertext bytes are affected? . . . 6
3.4 Constructing candidates for the last AES round key . . . 7
3.5 Intersecting observations . . . 7
3.6 Reversing AES-128 key expansion . . . 7

4 SM4: Complete Structure and First Round **8**
4.1 Official test vector . . . 8
4.2 SM4 round function . . . 8
4.3 SM4 key expansion . . . 8
4.4 Detailed computation of encryption round 0 . . . 9

5 Differential Fault Analysis of SM4: A Worked Last-Round Model **10**
5.1 Attack model and limitations . . . 10
5.2 Known ciphertext and hidden internal values . . . 10
5.3 Step 1: Inject a known XOR fault . . . 10
5.4 Step 2: Observe the S-box difference . . . 10
5.5 Step 3: Propagate the difference through L . . . 11
5.6 Step 4: Undo the linear layer from observed data . . . 11
5.7 Step 5: Convert S-box input candidates to key candidates . . . 11
5.8 Step 6: Use independent observations . . . 11
5.9 Step 7: Repeat for the other three bytes . . . 12
5.10 What happens after recovering the last subkey? . . . 12

6 SM4: Recovering the Master Key from Four Round Keys **13**
6.1 Required subkeys . . . 13
6.2 Reverse the recurrence . . . 13
6.3 Undo the initial constants . . . 13

7 Giraud Fault Attack on AES-128 and Its Adaptation to SM4 **14**
7.1 Core idea and comparison with Piret–Quisquater . . . 14
7.2 AES-128: precise fault location . . . 14
7.3 Worked AES-128 example: the first ciphertext byte . . . 14
7.4 Multiple faults and candidate intersection . . . 15
7.5 Adaptation of the single-bit idea to SM4 . . . 15
7.6 Worked SM4 example and recovery of rk_{31} . . . 16
7.7 From the final SM4 subkey to the master key . . . 17
7.8 Experimental implications . . . 17

8 AES versus SM4: Key Lessons **18**

9 Python Implementation and Reproducible Terminal Results **19**
9.1 Repository structure and execution . . . 19
9.2 AES-128 implementation: S-box, diffusion, and key expansion . . . 19
9.3 SM4 implementation: rotations, S-boxes, and 32 rounds . . . 21
9.4 AES Giraud: filtering last-round key candidates . . . 22
9.4.1 Observed AES terminal output . . . 22
9.5 SM4 Giraud-inspired filtering . . . 23
9.5.1 Observed SM4 terminal output . . . 24
9.6 Recovering SM4 rounds 31 through 28 . . . 25
9.7 Reversing the SM4 key schedule . . . 26
9.7.1 Independent terminal validation . . . 26
9.8 Interpretation of the results . . . 27

References and Scope **28**

Chapter 1

Purpose and Notation

This document reorganizes the supplied handwritten AES/SM4 notes into a progressive English explanation. It distinguishes established algorithmic facts, pedagogical fault simulations, and the additional work needed for a physical attack.

- $\oplus$: bitwise XOR; $\lll n$: 32-bit circular left rotation by n bits.
- S: an 8-bit substitution box (AES and SM4 use different S-boxes).
- C: correct ciphertext; F: faulty ciphertext; $\Delta = C \oplus F$.
- All hexadecimal words are written most-significant byte first. SM4 words are 32 bits.

Note. A known plaintext/key test vector is used to *verify* examples. In an actual key-recovery experiment, the attacker does not know the secret key.

Chapter 2

AES-128: Encryption from the Beginning

2.1 Block, key, and round structure

AES is a 128-bit block cipher. AES-128, AES-192, and AES-256 use 10, 12, and 14 rounds respectively. We focus on AES-128.

Plaintext	`00112233445566778899AABBCCDDEEFF`
Key K_0	`000102030405060708090A0B0C0D0E0F`
Ciphertext	`69C4E0D86A7B0430D8CDB78070B4C55A`

AES stores bytes in a 4×4 *column-major* state:

$$\begin{pmatrix} 00 & 44 & 88 & CC \\ 11 & 55 & 99 & DD \\ 22 & 66 & AA & EE \\ 33 & 77 & BB & FF \end{pmatrix}.$$

First apply AddRoundKey with K_0. Rounds 1–9 apply SubBytes, ShiftRows, MixColumns, AddRoundKey. Round 10 omits MixColumns.

2.2 Initial AddRoundKey

The state is XORed byte by byte with the first round key. For example, $\texttt{11} \oplus \texttt{01} = \texttt{10}$. The result is

$$\begin{pmatrix} 00 & 40 & 80 & C0 \\ 10 & 50 & 90 & D0 \\ 20 & 60 & A0 & E0 \\ 30 & 70 & B0 & F0 \end{pmatrix}.$$

2.3 SubBytes

Each byte is independently replaced through the AES S-box. Examples: $S(\texttt{00}) = \texttt{63}$, $S(\texttt{10}) = \texttt{CA}$, $S(\texttt{20}) = \texttt{B7}$, $S(\texttt{30}) = \texttt{04}$. The resulting state is

$$\begin{pmatrix} 63 & 09 & CD & BA \\ CA & 53 & 60 & 70 \\ B7 & D0 & E0 & E1 \\ 04 & 51 & E7 & 8C \end{pmatrix}.$$

The S-box is nonlinear: in general, $S(a \oplus b) \neq S(a) \oplus S(b)$.

2.4 ShiftRows

Rows 0, 1, 2, 3 rotate left by 0, 1, 2, 3 bytes respectively:

$$\begin{pmatrix} 63 & 09 & CD & BA \\ 53 & 60 & 70 & CA \\ E0 & E1 & B7 & D0 \\ 8C & 04 & 51 & E7 \end{pmatrix}.$$

2.5 MixColumns

Every column is multiplied over $GF(2^8)$ by

$$M = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}.$$

For the first input column $(\texttt{63}, \texttt{53}, \texttt{E0}, \texttt{8C})^T$,

$$b_0 = (02 \cdot 63) \oplus (03 \cdot 53) \oplus E0 \oplus 8C.$$

Multiplication is in the AES finite field, not ordinary integer arithmetic. For a separate illustrative column $(\texttt{DB}, \texttt{13}, \texttt{53}, \texttt{45})^T$, MixColumns yields $(\texttt{8E}, \texttt{4D}, \texttt{A1}, \texttt{BC})^T$. The first column of the official AES round-1 state becomes $(\texttt{5F}, \texttt{72}, \texttt{64}, \texttt{15})^T$.

2.6 Round keys

AES-128 expands the original 16-byte key into eleven 16-byte round keys $K_0, \ldots, K_{10}$. For this vector:

$$K_1 = \texttt{D6AA74FDD2AF72FADAA678F1D6AB76FE},$$

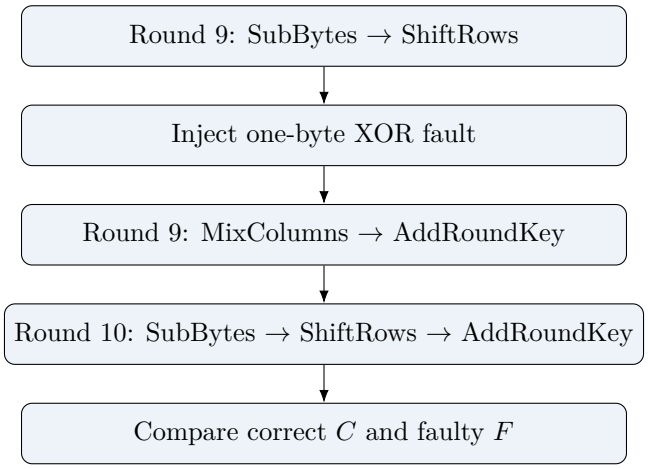
$$K_{10} = \texttt{13111D7FE3944A17F307A78B4D2B30C5}.$$

Chapter 3

Piret–Quisquater Differential Fault Analysis of AES

3.1 Where the fault is injected

In the classic model used in the notes, one nonzero byte is corrupted at the *input of MixColumns in round 9*, after SubBytes and ShiftRows. Both correct and faulty executions then proceed through round 9 and the final round.



3.2 Propagation through MixColumns

If the first byte of a column changes by e, the column difference after MixColumns is

$$\Delta_{MC} = (02e,\ e,\ e,\ 03e)^T.$$

For $e = \texttt{01}$, this is $(\texttt{02}, \texttt{01}, \texttt{01}, \texttt{03})^T$. AddRoundKey does not alter XOR differences because both executions use the same round key. The final SubBytes transforms these differences nonlinearly; ShiftRows moves them into separate ciphertext positions.

3.3 Which ciphertext bytes are affected?

For the four round-9 columns, the affected ciphertext positions (in corresponding row order) are:

Faulted column	Ciphertext byte positions
0	0, 13, 10, 7
1	4, 1, 14, 11
2	8, 5, 2, 15
3	12, 9, 6, 3

3.4 Constructing candidates for the last AES round key

For an affected ciphertext position j, and a guessed last-round key byte k, compute

$$d_j(k) = S^{-1}(C_j \oplus k) \oplus S^{-1}(F_j \oplus k).$$

Each byte has 256 candidate values. Four unknown key bytes have $256^4 = 2^{32}$ possible combinations before filtering. Construct a set $\mathcal{D}$ containing the MixColumns output differences for a single nonzero faulty byte in any of the four row positions:

$$\mathcal{D} = \{M(e\mathbf{e}_r) : r \in \{0, 1, 2, 3\},\ e \in \{1, \ldots, 255\}\}.$$

This produces $4 \cdot 255 = 1020$ four-byte difference patterns (not 1020 individual bytes). For each proposed key quartet, compute the four $d_j(k)$ values in their correct *pre-ShiftRows row order* and keep the quartet only if the resulting vector lies in $\mathcal{D}$.

3.5 Intersecting observations

A single correct/faulty pair typically leaves multiple candidate key quartets. Repeat with independent faults in the same column and intersect the candidate sets:

$$\mathcal{K}_{\text{surviving}} = \mathcal{K}_1 \cap \mathcal{K}_2 \cap \cdots .$$

Repeat across all four columns to recover the full 16-byte K_{10}. The notes illustrate candidate filtering using a hypothetical quartet (`13`, `2B`, `A7`, `17`); such pedagogical candidate examples should not be confused with a complete independently verified AES fault trace.

3.6 Reversing AES-128 key expansion

Let $w_0, \ldots, w_{43}$ be 32-bit expanded-key words. For $i \geq 4$,

$$w_i = \begin{cases} w_{i-4} \oplus g(w_{i-1}), & i \equiv 0 \pmod 4, \\ w_{i-4} \oplus w_{i-1}, & \text{otherwise}, \end{cases}$$

where $g(w) = \text{SubWord}(\text{RotWord}(w)) \oplus \text{Rcon}$. In reverse, compute w_{i-4} by XORing w_i with the same known right-hand transformation. Begin with $w_{40}, \ldots, w_{43}$ from K_{10} and work backward to $w_0, \ldots, w_3$, yielding K_0. No inverse S-box is required for this reversal because the relevant inputs to g are already known.

Chapter 4

SM4: Complete Structure and First Round

4.1 Official test vector

Plaintext	`0123456789ABCDEFFEDCBA9876543210`
Master key	`0123456789ABCDEFFEDCBA9876543210`
Ciphertext	`681EDF34D206965E86B3E94F536E4246`

Split the plaintext into four 32-bit words:

$$X_0 = \texttt{01234567}, \quad X_1 = \texttt{89ABCDEF}, \quad X_2 = \texttt{FEDCBA98}, \quad X_3 = \texttt{76543210}.$$

4.2 SM4 round function

SM4 performs 32 rounds, producing one new 32-bit word per round:

$$\boxed{X_{i+4} = X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i)}.$$

Write $T = L \circ \tau$, where τ substitutes the four bytes through the SM4 S-box and

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24).$$

The output reverses the final four words:

$$C = X_{35} \parallel X_{34} \parallel X_{33} \parallel X_{32}.$$

4.3 SM4 key expansion

The four initial key-schedule words are $K_i = MK_i \oplus FK_i$, with

$$FK_0 = \texttt{A3B1BAC6}, \quad FK_1 = \texttt{56AA3350},$$
$$FK_2 = \texttt{677D9197}, \quad FK_3 = \texttt{B27022DC}.$$

The key-schedule recurrence is

$$K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i), \quad rk_i = K_{i+4},$$

with $T' = L' \circ \tau$ and

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23).$$

Crucial distinction: encryption uses L, while key expansion uses L'. They have different rotation constants. For the first round, $rk_0 = \texttt{F12186F9}$.

4.4 Detailed computation of encryption round 0

Step 1: XOR the three newest words with the round key.

$$A = X_1 \oplus X_2 \oplus X_3 \oplus rk_0 = \texttt{F002C39E}.$$

Step 2: Apply the SM4 S-box to each byte.

$$\texttt{F0} \mapsto \texttt{18}, \quad \texttt{02} \mapsto \texttt{E9}, \quad \texttt{C3} \mapsto \texttt{92}, \quad \texttt{9E} \mapsto \texttt{B1}.$$

Thus $B = \tau(A) = \texttt{18E992B1}$.

Step 3: Apply the linear transformation.

Term	Hexadecimal value
B	18E992B1
$B \lll 2$	63A64AC4
$B \lll 10$	A64AC463
$B \lll 18$	4AC463A6
$B \lll 24$	B118E992
XOR of all five	26D99622

Step 4: XOR with the oldest word.

$$X_4 = X_0 \oplus L(B) = \texttt{01234567} \oplus \texttt{26D99622} = \boxed{\texttt{27FAD345}}.$$

The next four-word window is (X_1, X_2, X_3, X_4). The following words are $X_5 = \texttt{A18B4CB2}$, $X_6 = \texttt{11C1E22A}$, and $X_7 = \texttt{CC13E2EE}$. After 32 rounds and the output reversal, the official ciphertext is obtained.

Chapter 5

Differential Fault Analysis of SM4: A Worked Last-Round Model

5.1 Attack model and limitations

We use a *controlled, known XOR fault* in one byte of the input to the SM4 S-boxes in round 31. This is a pedagogical model, not a claim that a real clock/voltage glitch yields a known XOR value. In this model, all other round inputs remain unchanged. The last round is

$$X_{35} = X_{31} \oplus L(\tau(A)), \qquad A = X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}.$$

The attacker observes the correct ciphertext and a faulty ciphertext. The secret internal values below are supplied *only for checking the simulation*.

5.2 Known ciphertext and hidden internal values

$$X_{35} = \texttt{681EDF34}, \quad X_{34} = \texttt{D206965E},$$
$$X_{33} = \texttt{86B3E94F}, \quad X_{32} = \texttt{536E4246}.$$

For this official test vector, the simulation has

$$X_{31} = \texttt{7B938F4C}, \quad A = \texttt{96FF9D45}, \quad rk_{31} = \texttt{9124A012}.$$

The attacker must recover the key rather than assume it.

5.3 Step 1: Inject a known XOR fault

Alter the first byte of A by $e = \texttt{01}$:

$$A = \texttt{96FF9D45} \longrightarrow A' = \texttt{97FF9D45}.$$

5.4 Step 2: Observe the S-box difference

The two S-box outputs are

$$B = \tau(A) = \texttt{1A488C73}, \qquad B' = \tau(A') = \texttt{55488C73}.$$

Consequently,

$$\Delta B = B \oplus B' = \texttt{4F000000}.$$

5.5 Step 3: Propagate the difference through L

Because L is XOR-linear,

$$\Delta X_{35} = L(B) \oplus L(B') = L(\Delta B) = \texttt{734E3D3D}.$$

The correct last word is $X_{35} = \texttt{681EDF34}$, so the faulty last word is

$$X'_{35} = \texttt{681EDF34} \oplus \texttt{734E3D3D} = \texttt{1B50E209}.$$

Therefore the attacker sees

Correct ciphertext	681EDF34	D206965E	86B3E94F	536E4246
Faulty ciphertext	1B50E209	D206965E	86B3E94F	536E4246
Difference	734E3D3D	00000000	00000000	00000000

5.6 Step 4: Undo the linear layer from observed data

The attacker computes

$$\Delta B = L^{-1}(\Delta X_{35}) = L^{-1}(\texttt{734E3D3D}) = \texttt{4F000000}.$$

The first S-box byte therefore satisfies

$$S(a) \oplus S(a \oplus \texttt{01}) = \texttt{4F}.$$

Enumerating all 256 possible input bytes gives the candidate set

$$\mathcal{A}_1 = \{\texttt{96}, \texttt{97}\}.$$

5.7 Step 5: Convert S-box input candidates to key candidates

The attacker knows the last three words from the ciphertext:

$$X_{32} \oplus X_{33} \oplus X_{34} = \texttt{07DB3D57}.$$

Since $rk_{31} = A \oplus \texttt{07DB3D57}$, the first byte of the key is

$$rk_{31}[0] = a \oplus \texttt{07}.$$

Thus $a = \texttt{96}$ yields $\texttt{91}$, while $a = \texttt{97}$ yields $\texttt{90}$. After one observation, the candidates are $\{\texttt{91}, \texttt{90}\}$.

5.8 Step 6: Use independent observations

Repeat with the *same plaintext and state*, but use a known XOR fault $e = \texttt{02}$. The new S-box difference is $d_2 = \texttt{81}$. Keep only input candidates satisfying both

$$\begin{cases} S(a) \oplus S(a \oplus \texttt{01}) = \texttt{4F}, \\ S(a) \oplus S(a \oplus \texttt{02}) = \texttt{81}. \end{cases}$$

The intersection identifies $a = \texttt{96}$, hence $rk_{31}[0] = \texttt{91}$.

5.9 Step 7: Repeat for the other three bytes

The supplied worked example reports the following bytewise observations under the same controlled-fault assumption:

Byte	d for $e = 01$	d for $e = 02$	Recovered A	Recovered rk_{31}
0	4F	81	96	91
1	71	83	FF	24
2	79	6F	9D	A0
3	80	C9	45	12

This reconstructs

$$A = \texttt{96FF9D45}, \qquad rk_{31} = A \oplus \texttt{07DB3D57} = \boxed{\texttt{9124A012}}.$$

Note. This illustrates last-round subkey recovery *conditional on the fault model*. The listed observations should be validated against a software implementation before treating them as experimental results.

5.10 What happens after recovering the last subkey?

Knowing rk_{31}, reconstruct the previous state word:

$$X_{31} = X_{35} \oplus T(X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}).$$

For the example, $X_{31} = \texttt{7B938F4C}$. However, this does *not* automatically reveal rk_{30}. Additional fault observations and state constraints are required for rounds 30, 29, and 28. The present worked example does not demonstrate that stage of the attack.

Chapter 6

SM4: Recovering the Master Key from Four Round Keys

6.1 Required subkeys

If four consecutive final subkeys have been recovered, then

Key-schedule word	Round key
K_{32}	$rk_{28} =$ `428D3654`
K_{33}	$rk_{29} =$ `62293496`
K_{34}	$rk_{30} =$ `01CF72E5`
K_{35}	$rk_{31} =$ `9124A012`

These are reference subkeys from the official example; *only* rk_{31} was recovered in the preceding pedagogical DFA walkthrough.

6.2 Reverse the recurrence

The forward key schedule is

$$K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i).$$

XOR is self-inverse, so

$$\boxed{K_i = K_{i+4} \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)}.$$

Start with $i = 31$, then decrease to $i = 0$. For example,

$$K_{31} = K_{35} \oplus T'(K_{32} \oplus K_{33} \oplus K_{34} \oplus CK_{31}),$$

where $CK_{31} =$ `646B7279`. This yields the previous expanded-key word $K_{31} =$ `F1780C81` in the official test vector. Continue to recover K_0, K_1, K_2, K_3.

6.3 Undo the initial constants

Finally,

$$MK_i = K_i \oplus FK_i \qquad (i = 0, 1, 2, 3),$$

and concatenate the four master-key words:

$$\boxed{MK = \texttt{0123456789ABCDEFFEDCBA9876543210}}.$$

Chapter 7

Giraud Fault Attack on AES-128 and Its Adaptation to SM4

7.1 Core idea and comparison with Piret–Quisquater

Giraud's AES fault-analysis approach exploits a *single-bit* fault close to the last round. In contrast to the Piret–Quisquater example, which injects a byte fault before the ninth-round MixColumns and typically modifies four ciphertext bytes, a single-byte fault immediately before the final SubBytes affects only one ciphertext byte: the last AES round does not contain MixColumns. The attacker tests last-round key-byte candidates by checking whether the inverse-S-box input difference has Hamming weight one.

7.2 AES-128: precise fault location

Let U denote the state after round 9 AddRoundKey, just before round 10 SubBytes. A one-bit XOR fault modifies a single input byte a into $a' = a \oplus e$, where $e \in \{\texttt{01}, \texttt{02}, \texttt{04}, \texttt{08}, \texttt{10}, \texttt{20}, \texttt{40}, \texttt{80}\}$. Round 10 performs SubBytes, ShiftRows and AddRoundKey K_{10}; there is no MixColumns. ShiftRows relocates the changed byte without increasing the number of affected bytes.

For the affected ciphertext position j, with correct byte c, faulty byte f, and unknown last-round key byte k, the attacker evaluates

$$\delta(k) = S_{\mathrm{AES}}^{-1}(c \oplus k) \oplus S_{\mathrm{AES}}^{-1}(f \oplus k). \tag{7.1}$$

The candidate is compatible with the fault model if

$$\delta(k) \in \mathcal{E} = \{\texttt{01}, \texttt{02}, \texttt{04}, \texttt{08}, \texttt{10}, \texttt{20}, \texttt{40}, \texttt{80}\}. \tag{7.2}$$

Equivalently, the 8-bit value $\delta(k)$ must have Hamming weight exactly one. If the injected bit position is known, the condition can be made stronger: $\delta(k) = e$.

7.3 Worked AES-128 example: the first ciphertext byte

We use the official AES-128 test vector already introduced in this report:

$$P = \texttt{00112233445566778899AABBCCDDEEFF},$$
$$K_0 = \texttt{000102030405060708090A0B0C0D0E0F},$$
$$C = \texttt{69C4E0D86A7B0430D8CDB78070B4C55A}.$$

The first byte entering the final SubBytes is $a = \texttt{BD}$. Inject $e = \texttt{01}$ in that byte:

$$\texttt{BD} \oplus \texttt{01} = \texttt{BC}. \tag{7.3}$$

The AES S-box maps $S(\texttt{BD}) = \texttt{7A}$ and $S(\texttt{BC}) = \texttt{65}$. The first byte of K_{10} is $\texttt{13}$, so

$$c = \texttt{7A} \oplus \texttt{13} = \texttt{69},$$
$$f_1 = \texttt{65} \oplus \texttt{13} = \texttt{76}.$$

Thus the simulated faulty ciphertext is

$$F_1 = \texttt{76C4E0D86A7B0430D8CDB78070B4C55A}. \tag{7.4}$$

The attacker does not assume the value $\texttt{13}$: it is shown here solely to verify the example. For candidate $k = \texttt{13}$, equation (7.1) gives

$$S^{-1}(\texttt{69} \oplus \texttt{13}) = S^{-1}(\texttt{7A}) = \texttt{BD},$$
$$S^{-1}(\texttt{76} \oplus \texttt{13}) = S^{-1}(\texttt{65}) = \texttt{BC},$$
$$\delta(\texttt{13}) = \texttt{BD} \oplus \texttt{BC} = \boxed{\texttt{01}}.$$

The candidate is compatible with a single-bit fault, but one correct/faulty pair may leave several candidates.

7.4 Multiple faults and candidate intersection

Repeat the encryption with the same plaintext and key and introduce an independent single-bit fault at the same location. In the worked example, choosing $e_2 = \texttt{02}$ produces $a'_2 = \texttt{BF}$ and a second faulty first ciphertext byte $f_2 = \texttt{1B}$. For observation t, define

$$\mathcal{K}_t = \{k \in \{0, \ldots, 255\} : S^{-1}(c \oplus k) \oplus S^{-1}(f_t \oplus k) \in \mathcal{E}\}. \tag{7.5}$$

The surviving candidates are the intersection $\mathcal{K} = \bigcap_t \mathcal{K}_t$. For the illustrated pair of faults, the recovered first byte is $K_{10}[0] = \texttt{13}$. The number of observations required is not fixed: it depends on the fault model and the observed differences.

Repeating the experiment across all sixteen pre-final-SubBytes byte positions allows the attacker, under this model, to reconstruct all sixteen bytes of K_{10}. For the official vector,

$$K_{10} = \texttt{13111D7FE3944A17F307A78B4D2B30C5}. \tag{7.6}$$

The AES-128 key schedule can then be reversed to recover the original key K_0, as explained in the preceding AES key-expansion chapter.

7.5 Adaptation of the single-bit idea to SM4

The corresponding SM4 experiment is an *adaptation of the single-bit differential filtering principle*, rather than a claim that the original AES attack transfers unchanged. The SM4 last round is

$$X_{35} = X_{31} \oplus L(\tau(A)), \qquad A = X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}. \tag{7.7}$$

The output is $C = (X_{35}, X_{34}, X_{33}, X_{32})$. Inject a single-bit fault in one byte of A, just before the last-round S-boxes. Unlike AES, the SM4 linear map L diffuses the one-byte S-box output difference over the 32-bit word X_{35}. The other three ciphertext words remain unchanged *under the assumed isolated-fault model.*

Let F be the faulty ciphertext and $\Delta X_{35} = X_{35} \oplus X'_{35}$. Since L is linear and invertible, the attacker can compute

$$\Delta B = L^{-1}(\Delta X_{35}) = \tau(A) \oplus \tau(A'). \tag{7.8}$$

For an affected byte with S-box input a and unknown single-bit error e, candidate inputs satisfy

$$S_{\mathrm{SM4}}(a) \oplus S_{\mathrm{SM4}}(a \oplus e) = d, \quad e \in \mathcal{E}, \tag{7.9}$$

where d is the corresponding byte of ΔB. If e is known from the simulation, it is used directly; otherwise all eight one-bit masks must be tested. Multiple correct/faulty observations reduce the candidate set by intersection.

7.6 Worked SM4 example and recovery of rk_{31}

Using the official SM4 test vector,

$$\begin{aligned} P &= \texttt{0123456789ABCDEFFEDCBA9876543210}, \\ MK &= \texttt{0123456789ABCDEFFEDCBA9876543210}, \\ C &= \texttt{681EDF34D206965E86B3E94F536E4246}, \end{aligned}$$

the simulated input of the last S-box layer is $A = \texttt{96FF9D45}$. A one-bit fault $e = \texttt{01000000}$ gives $A' = \texttt{97FF9D45}$. The substitution outputs are

$$\begin{aligned} \tau(A) &= \texttt{1A488C73}, \\ \tau(A') &= \texttt{55488C73}, \\ \Delta B &= \texttt{4F000000}. \end{aligned}$$

After the linear layer,

$$\Delta X_{35} = L(\texttt{4F000000}) = \texttt{734E3D3D}. \tag{7.10}$$

Therefore the faulty output is

$$F_1 = \texttt{1B50E209D206965E86B3E94F536E4246}. \tag{7.11}$$

From the observed difference alone, the attacker obtains $\Delta B = L^{-1}(\texttt{734E3D3D}) = \texttt{4F000000}$. With known fault mask `01` in the first byte, the candidate equation is

$$S_{\mathrm{SM4}}(a) \oplus S_{\mathrm{SM4}}(a \oplus \texttt{01}) = \texttt{4F}. \tag{7.12}$$

The first observation permits $a \in \{\texttt{96}, \texttt{97}\}$. A second simulated fault of mask `02` gives the first-byte S-box difference `81`. Intersecting both sets identifies $a = \texttt{96}$.

The three known ciphertext words yield

$$X_{32} \oplus X_{33} \oplus X_{34} = \texttt{07DB3D57}. \tag{7.13}$$

Thus the first last-round key byte is

$$rk_{31}[0] = \texttt{96} \oplus \texttt{07} = \boxed{\texttt{91}}. \tag{7.14}$$

The supplied worked example applies analogous observations to the remaining three S-box inputs, obtaining

$$A = \texttt{96FF9D45}, \qquad rk_{31} = A \oplus \texttt{07DB3D57} = \boxed{\texttt{9124A012}}. \tag{7.15}$$

The bytewise simulated differences are summarized below.

Byte	d for $e = 01$	d for $e = 02$	Recovered A	Recovered rk_{31}
0	4F	81	96	91
1	71	83	FF	24
2	79	6F	9D	A0
3	80	C9	45	12

Note. The numerical values in this subsection are simulated, not physical fault-injection measurements. In particular, knowledge of the exact XOR mask is a strong experimental assumption.

7.7 From the final SM4 subkey to the master key

Recovering rk_{31} is not equivalent to recovering the 128-bit master key. The SM4 key schedule needs four consecutive 32-bit round keys to be inverted. In the official test vector these are

$$rk_{28} = \texttt{428D3654}, \qquad rk_{29} = \texttt{62293496},$$
$$rk_{30} = \texttt{01CF72E5}, \qquad rk_{31} = \texttt{9124A012}.$$

The values rk_{28}, rk_{29} and rk_{30} are *reference values*, not keys shown to have been recovered by the two last-round faults. Additional experiments and appropriate internal-state reconstruction would be necessary to obtain them.

With four consecutive round keys, set $K_{i+4} = rk_i$ and invert the SM4 recurrence:

$$K_i = K_{i+4} \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i). \tag{7.16}$$

Iterating backward yields K_0, K_1, K_2, K_3. Undo the fixed initial constants FK_i via $MK_i = K_i \oplus FK_i$ and concatenate the four words to recover the 128-bit master key.

7.8 Experimental implications

A software simulation can inject an exact XOR mask at a chosen intermediate value. On an STM32 target, a voltage or clock glitch may instead alter multiple bits, skip an instruction, affect control flow, or produce no useful fault. A practical evaluation should therefore log correct and faulty ciphertexts, classify observed fault patterns, verify whether the assumed location and single-bit model are plausible, and report recovery success separately from simulation-only results.

Chapter 8

AES versus SM4: Key Lessons

Property	AES-128	SM4
Block size	128 bits	128 bits
Key size	128 bits	128 bits
Rounds	10	32
State	4×4 byte matrix	Four 32-bit words
Nonlinearity	AES S-box	SM4 S-box
Linear diffusion	MixColumns and ShiftRows	Rotations and XOR in L
Last-round target	K_{10}, 128 bits	rk_{31}, 32 bits
Master-key recovery	Reverse AES expansion	Reverse SM4 expansion from four consecutive round keys

Summary

For AES Giraud, inject a one-bit fault before final-round SubBytes, test inverse-S-box differences for Hamming weight one, and intersect key-byte candidates. The corresponding SM4 adaptation first inverts the linear map L to isolate the S-box differential.

For AES Piret–Quisquater, compare correct and faulty ciphertexts, invert the last S-box under candidate K_{10} bytes, match round-9 MixColumns differential patterns, intersect candidates, and reverse the AES key schedule.

For the illustrated SM4 model, compare correct and faulty ciphertexts, invert the linear map L, test S-box input candidates using the known fault differences, derive rk_{31}, and obtain further subkeys with additional appropriate experiments. Once four consecutive round keys are known, the SM4 key schedule can be reversed exactly.

Chapter 9

Python Implementation and Reproducible Terminal Results

This chapter connects the preceding mathematical derivations to the actual Python project provided with this report. The listings below are *excerpts of the project source files*, not pseudocode. The console blocks were captured by executing the supplied scripts on 24 October 2026. These are software simulations with a controllable fault observer, not physical STM32 measurements.

9.1 Repository structure and execution

The principal modules are `aes.py` (AES operations), `sm4.py` (SM4 operations), `piret.py` (DFA and round peeling), `giraud.py` (single-bit candidate filtering), and `run.py` (combined demonstration). All five source files are included in the Overleaf project archive.

```
python giraud.py --algorithm aes --known-bit
python giraud.py --algorithm sm4 --known-bit
python piret.py --algorithm sm4
```

The project also supports an unknown-bit model by omitting `-known-bit`. The `-known-bit` option is used for the reproducible outputs reproduced here.

9.2 AES-128 implementation: S-box, diffusion, and key expansion

The following code implements AES finite-field multiplication and column mixing. This is the operation responsible for the characteristic four-byte differential exploited by Piret–Quisquater.

Listing 9.1: AES finite-field arithmetic, from aes.py

```
def xtime(a):
    """Multiply a byte by x (02) in GF(2^8), reducing by polynomial 0x11B."""
    result = a << 1

    # If the x^8 term overflows, reduction contributes 0x1B to the low byte.
    if a & 0x80:
        result ^= 0x1B

    return result & 0xFF


def gf_mul(a, b):
    """Multiply bytes in the AES field; field addition is XOR, not integer +."""
    result = 0

    for _ in range(8):
        if b & 1:
```

```
            result ^= a

        # Shift-and-add multiplication, using xtime for polynomial reduction.
        a = xtime(a)
        b >>= 1

    return result


# ================================================================================
# 5. ROUND TRANSFORMATIONS: ALL MATRIX OPERATIONS ARE IN PLACE
# ================================================================================
```

Listing 9.2: AES MixColumns, from aes.py

```
def mix_column(column):
    """Return M * column in GF(2^8), without modifying the four input bytes.

    Copying a column before mixing matters: every output byte must use the same
    original input, never a value already overwritten earlier in the operation.
    """
    result = []
    for coefficients in MIX_MATRIX:
        value = 0
        for coefficient, byte in zip(coefficients, column):
            value ^= gf_mul(coefficient, byte)
        result.append(value)
    return result


def mix_columns(state):
    """Mix each column in place; the normal algorithm never prints output."""
    for column in range(4):
        mixed = mix_column([state[row][column] for row in range(4)])
        for row in range(4):
            state[row][column] = mixed[row]
```

Listing 9.3: AES round-key expansion, from aes.py

```
def key_expansion(key):
    """Expand the 16-byte master key into eleven independent round-key matrices.

    Each new word XORs the word four positions back with the previous word.
    Every fourth word first applies RotWord, SubWord and the round constant.
    """
    validate_block(key, "AES-128 key")
    # Four initial words become 44 words: K0 plus ten encryption round keys.
    words = [list(key[4 * i:4 * i + 4]) for i in range(4)]

    for i in range(4, 4 * (N_ROUNDS + 1)):
        temp = words[i - 1].copy()

        if i % 4 == 0:
            # Rcon changes only the first byte of the transformed key word.
            temp = sub_word(rot_word(temp))
            temp[0] ^= RCON[i // 4]

        new_word = [words[i - 4][j] ^ temp[j] for j in range(4)]

        words.append(new_word)

    round_keys = []

    for round_number in range(N_ROUNDS + 1):
        # Convert four consecutive words into the same layout as the data state.
        key_bytes = []

        for i in range(4):
            key_bytes.extend(words[round_number * 4 + i])

        round_keys.append(bytes_to_state(key_bytes))

```

```
    return round_keys


# ============================================================================
# 7. NORMAL ENCRYPTION / DECRYPTION AND OPTIONAL EXTERNAL OBSERVATION
# ============================================================================
```

9.3 SM4 implementation: rotations, S-boxes, and 32 rounds

The SM4 code keeps the encryption diffusion L distinct from the key-schedule diffusion L'. The observer hook allows the test harness to modify an intermediate value at a chosen round.

Listing 9.4: SM4 rotations, diffusion, and substitution, from sm4.py

```
def rotl(val: int, count: int) -> int:
    """Rotate a 32-bit unsigned word left; overflow wraps into the low bits."""
    return ((val << count) & WORD_MASK) | (val >> (32 - count))


def linear(b: int) -> int:
    """Data diffusion L: XOR the word with rotations by 2, 10, 18 and 24.

    L is XOR-linear: L(a XOR b) = L(a) XOR L(b). The SM4 DFA uses this
    property to invert an observed output difference before testing the S-box.
    """
    return b ^ rotl(b, 2) ^ rotl(b, 10) ^ rotl(b, 18) ^ rotl(b, 24)


def linear_key(b: int) -> int:
    """Key-schedule diffusion L-prime: rotations 13 and 23, not data-map L."""
    return b ^ rotl(b, 13) ^ rotl(b, 23)


def sbox(a: int) -> int:
    """Apply tau: substitute all four bytes, preserving big-endian word order."""
    input_bytes = struct.pack('>I', a)
    substituted_bytes = bytes(SBOX[byte] for byte in input_bytes)
    return struct.unpack('>I', substituted_bytes)[0]


# ============================================================================
# 4. SINGLE-ROUND RECURRENCES
# ============================================================================
```

Listing 9.5: SM4 round function and round-key generation, from sm4.py

```
def round(x: list[int], rk: int, observer=None, round_index=0):
    """Return the next rolling window without modifying the supplied words.

    A = X[i+1] XOR X[i+2] XOR X[i+3] XOR rk[i]
    X[i+4] = X[i] XOR L(tau(A))
    """
    sbox_input = observe(observer, round_index, "Sbox input", x[1] ^ x[2] ^ x[3] ^ rk)
    substituted = observe(observer, round_index, "Sbox output", sbox(sbox_input))
    diffused = observe(observer, round_index, "Linear output", linear(substituted))
    new_word = observe(observer, round_index, "New word", x[0] ^ diffused)
    return x[1], x[2], x[3], new_word


def generate_round_key(k: list[int], round: int) -> int:
    """Compute K[i+4] from four consecutive schedule words and CK[i].

    The parameter name round is retained for callers using keyword arguments.
    Encryption and expansion share tau, but expansion must use linear_key.
    """
    return k[0] ^ linear_key(sbox(k[1] ^ k[2] ^ k[3] ^ CK[round]))


# ============================================================================
# 5. KEY EXPANSION: FOUR MASTER WORDS TO 32 ROUND KEYS
```

```
# ============================================================================


def key_expansion(key: bytes) -> list[int]:
    """Return rk0..rk31; each is one new word from the key recurrence."""
    if len(key) != BLOCK_SIZE:
        raise ValueError("SM4 key must contain exactly 16 bytes.")
    master_words = struct.unpack(">4I", key)
    schedule_words = [master_words[i] ^ FK[i] for i in range(4)]

    for i in range(N_ROUNDS):
        # Keep previous words for readability; the recurrence consumes a window.
        next_key = generate_round_key(schedule_words[i:i+4], i)
        schedule_words.append(next_key)

    # The initial four whitened words are not encryption round keys.
    return schedule_words[4:]


# ============================================================================
# 6. SHARED BLOCK PROCESSING AND PUBLIC ENCRYPT/DECRYPT WRAPPERS
# ============================================================================
```

9.4 AES Giraud: filtering last-round key candidates

For each possible final-round key byte, the implementation undoes the final AddRoundKey and S-box, and tests whether the recovered input difference is a single bit. The solver does not receive the master key.

Listing 9.6: AES Giraud candidate test, from giraud.py

```
def aes_candidates(correct, faulty, known_mask=None):
    """Return the affected output position and its compatible K10 byte values.

    For each guess k:
      delta = InvSbox[C[j] XOR k] XOR InvSbox[F[j] XOR k]
    Keep k when delta has exactly one set bit, or equals the known bit mask.
    """
    aes.validate_block(correct, "Correct ciphertext")
    aes.validate_block(faulty, "Faulty ciphertext")
    if known_mask is not None:
        validate_mask(known_mask)

    # Without final-round MixColumns, only one ciphertext byte should change.
    changed = [index for index in range(16) if correct[index] != faulty[index]]
    if len(changed) != 1:
        raise ValueError("AES Giraud requires exactly one changed ciphertext byte.")
    position = changed[0]
    allowed = SINGLE_BITS if known_mask is None else (known_mask,)

    candidates = set()
    for guess in range(256):
        clean_input = AES_INVERSE_SBOX[correct[position] ^ guess]
        faulty_input = AES_INVERSE_SBOX[faulty[position] ^ guess]
        difference = clean_input ^ faulty_input
        if difference in allowed:
            candidates.add(guess)
    return position, candidates
```

9.4.1 Observed AES terminal output

The following excerpt is from an actual execution of `python giraud.py -algorithm aes -known-bit`. It shows the filtering process and final verification.

Listing 9.7: Recorded AES Giraud execution excerpt

```

```

```
================================================================================
GIRAUD / AES-128
Correct ciphertext: 69C4E0D86A7B0430D8CDB78070B4C55A
Bit known to solver: True
AES: require HW(InvSbox[C XOR k] XOR InvSbox[F XOR k]) = 1.

Key byte 0, observation 1, simulator mask 01
Correct: 69 C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 B4 C5 5A
Faulty: 76 C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 B4 C5 5A
XOR: 1F 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Candidate count: 256 -> 4
Survivors: 0A 0C 13 15

Key byte 0, observation 2, simulator mask 02
Correct: 69 C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 B4 C5 5A
Faulty: 1B C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 B4 C5 5A
XOR: 72 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Candidate count: 4 -> 1
Survivors: 13

Key byte 13, observation 1, simulator mask 01
Correct: 69 C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 B4 C5 5A
Faulty: 69 C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 83 C5 5A
XOR: 00 00 00 00 00 00 00 00 00 00 00 00 00 37 00 00
Candidate count: 256 -> 2
Survivors: 1C 2B

Key byte 13, observation 2, simulator mask 02
Correct: 69 C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 B4 C5 5A
Faulty: 69 C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 7B C5 5A
XOR: 00 00 00 00 00 00 00 00 00 00 00 00 00 CF 00 00

... [intermediate observations omitted from this printed excerpt]

XOR: 00 00 00 9A 00 00 00 00 00 00 00 00 00 00 00 00
Candidate count: 2 -> 1
Survivors: 7F

Recovered K10: 13111D7FE3944A17F307A78B4D2B30C5
Recovered master key: 000102030405060708090A0B0C0D0E0F
Verification: PASS / 32 faulty outputs
```

9.5 SM4 Giraud-inspired filtering

SM4 requires an extra step: invert the linear transformation L to isolate the S-box output difference. The following code tests candidate S-box inputs, then translates them into candidates for a byte of the last-round subkey using the three unchanged state words.

Listing 9.8: SM4 S-box differential candidate filter, from giraud.py

```python
def sm4_candidates(correct, faulty, known_mask=None):
    """Return affected byte, round-key guesses, input guesses and DeltaB.

    Inputs may be actual final ciphertexts or peeled views at an earlier round.

    1. Observe DeltaX35 from the first output word.
    2. Undo linear diffusion: DeltaB = L-inverse(DeltaX35).
    3. Keep a if S(a) XOR S(a XOR e) matches the observed byte difference.
    4. Convert a into a key guess using the three unchanged output words.
    """
    aes.validate_block(correct, "Correct ciphertext")
    aes.validate_block(faulty, "Faulty ciphertext")
    if known_mask is not None:
        validate_mask(known_mask)

    # SM4 outputs X35 || X34 || X33 || X32; only X35 may change in this model.
    if correct[4:] != faulty[4:]:
        raise ValueError("SM4 last-round model requires three output words unchanged.")
    delta_x35 = int.from_bytes(correct[:4], "big") ^ int.from_bytes(faulty[:4], "big")
    delta_b = inverse_linear(delta_x35)
```

```
    differences = delta_b.to_bytes(4, "big")
    changed = [index for index in range(4) if differences[index] != 0]
    if len(changed) != 1:
        raise ValueError("SM4 inverse-L difference must affect one S-box byte.")
    position = changed[0]
    allowed = SINGLE_BITS if known_mask is None else (known_mask,)

    # The input value a is unknown. Test every value against each permitted bit.
    inputs = set()
    for guess in range(256):
        for error in allowed:
            difference = sm4.SBOX[guess] ^ sm4.SBOX[guess ^ error]
            if difference == differences[position]:
                inputs.add(guess)
                break

    # A = X32 XOR X33 XOR X34 XOR rk31; all three data words are observable.
    x34 = int.from_bytes(correct[4:8], "big")
    x33 = int.from_bytes(correct[8:12], "big")
    x32 = int.from_bytes(correct[12:16], "big")
    known_byte = ((x32 ^ x33 ^ x34) >> (24 - 8 * position)) & 0xFF
    candidates = {guess ^ known_byte for guess in inputs}
    return position, candidates, inputs, delta_b
```

9.5.1 Observed SM4 terminal output

The first fault produces a 32-bit ciphertext difference; the inverse-linear operation reduces it to a one-byte S-box difference. A second independent observation reduces the key-byte candidates from two to one.

Listing 9.9: Recorded SM4 Giraud-inspired execution excerpt

```

================================================================================
GIRAUD-INSPIRED / SM4
Correct ciphertext: 681EDF34D206965E86B3E94F536E4246
Bit known to solver: True
SM4: invert L, then test S(a) XOR S(a XOR e) for one-bit errors.

Round 31, key byte 0, observation 1
Correct: 68 1E DF 34 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
Faulty: 1B 50 E2 09 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
XOR: 73 4E 3D 3D 00 00 00 00 00 00 00 00 00 00 00 00
Undo recovered later rounds: {}
Correct: 68 1E DF 34 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
Faulty: 1B 50 E2 09 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
XOR: 73 4E 3D 3D 00 00 00 00 00 00 00 00 00 00 00 00
Candidates: 256 -> 2

Round 31, key byte 0, observation 2
Correct: 68 1E DF 34 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
Faulty: ED 9D D9 32 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
XOR: 85 83 06 06 00 00 00 00 00 00 00 00 00 00 00 00
Undo recovered later rounds: {}
Correct: 68 1E DF 34 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
Faulty: ED 9D D9 32 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
XOR: 85 83 06 06 00 00 00 00 00 00 00 00 00 00 00 00
Candidates: 2 -> 1

Round 31, key byte 1, observation 1
Correct: 68 1E DF 34 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
Faulty: AD AB AF F1 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
XOR: C5 B5 70 C5 00 00 00 00 00 00 00 00 00 00 00 00
Undo recovered later rounds: {}

... [intermediate observations omitted from this printed excerpt]

Candidates: 2 -> 1
Recovered rk31: 9124A012
Recovered rk30: 01CF72E5
Recovered rk29: 62293496
```

```
Recovered rk28: 428D3654
Recovered master key: 0123456789ABCDEFFEDCBA9876543210
Verification: PASS / 32 faulty outputs / no reference subkeys
```

9.6 Recovering SM4 rounds 31 through 28

The source project implements an additional *multi-round simulated attack*, beyond the last-round example described earlier in this report. Once rk_{31} is recovered, its value is used to undo the last round in both correct and faulty ciphertexts. A separate controlled fault at round 30 can then be analyzed as a last-round differential on the reconstructed state. The same process is repeated for rounds 29 and 28.

Crucially, the algorithm does *not* supply reference round keys to the recovery function. It uses previously recovered keys to peel later rounds. However, its fault oracle must be capable of injecting faults at each of the four targeted rounds, a strong capability not established by these software tests on physical hardware.

Listing 9.10: Undoing already recovered SM4 rounds, from piret.py

```python
def peel_sm4(ciphertext, target_round, recovered_keys):
    """Undo only rounds whose subkeys have ALREADY been recovered by DFA.

    Return a reversed four-word output at the end of target_round. This lets
    the same last-round equations analyze rounds 30, 29 and 28 in succession.
    No reference round keys or secret intermediate values are accepted.
    """
    aes.validate_block(ciphertext, "SM4 ciphertext")
    if not 28 <= target_round <= 31:
        raise ValueError("Full SM4 recovery targets rounds 28..31.")
    words = list(struct.unpack(">4I", ciphertext))[::-1]
    for number in range(31, target_round, -1):
        if number not in recovered_keys:
            raise ValueError(f"Cannot undo round {number} before recovering its subkey.")
        previous = words[3] ^ sm4.linear(sm4.sbox(words[0] ^ words[1] ^ words[2] ^
             recovered_keys[number]))
        words = [previous, words[0], words[1], words[2]]
    return struct.pack(">4I", *words[::-1])
```

Listing 9.11: Sequential recovery of four SM4 subkeys, from piret.py

```python
def recover_sm4_master_key(correct, fault_oracle, masks=(1, 2, 3, 4, 5, 6, 7, 8), start_byte=0,
     candidate_filter=None, observer=None):
    """Recover rk31 -> rk28 and reverse the schedule, using outputs only.

    fault_oracle(round, byte, mask) returns a faulty ciphertext. It represents
    a target capable of a controlled fault in EACH of the final four rounds.
    The solver receives no master key, schedule, or reference subkeys.
    The default filter uses known masks; Giraud supplies its single-bit filter.
    """
    aes.validate_block(correct, "Correct SM4 ciphertext")
    masks = tuple(masks)
    if not masks or len(set(masks)) != len(masks) or any(not 1 <= mask <= 255 for mask in masks) or
         not 0 <= start_byte < 4:
        raise ValueError("Use distinct nonzero byte masks and start_byte 0..3.")
    keys, history, counts, used = {}, [], {}, 0
    for number in range(31, 27, -1):
        clean_view = peel_sm4(correct, number, keys)
        subkey = bytearray(4)
        for byte in [(start_byte + offset) % 4 for offset in range(4)]:
            survivors = None
            for observation, mask in enumerate(masks, 1):
                faulty = bytes(fault_oracle(number, byte, mask))
                faulty_view = peel_sm4(faulty, number, keys)
                if candidate_filter is None:
                    fresh, inputs, delta = sm4_byte_candidates(clean_view, faulty_view, byte, mask)
                else:
                    position, fresh, inputs, delta = candidate_filter(clean_view, faulty_view, mask)
```

```
                    if position != byte:
                        raise ValueError("Fault observation does not match the requested byte.")
                before = 256 if survivors is None else len(survivors)
                survivors = set(fresh) if survivors is None else survivors & fresh
                if not survivors:
                    raise ValueError("SM4 candidate intersection is empty; fault model is
                        incompatible.")
                used += 1
                counts[f"{number}:{byte}"] = len(survivors)
                event = dict(round=number, byte=byte, observation=observation, mask=mask,
                    before=before, pair_candidates=len(fresh), remaining=len(survivors),
                    survivors=sorted(survivors), inputs=sorted(inputs), delta_b=delta,
                    correct=correct.hex(), faulty=faulty.hex(), clean_view=clean_view.hex(),
                    faulty_view=faulty_view.hex(), peeled_with={str(i): f"{value:08X}" for i, value in
                    keys.items()})
                history.append(event)
                if observer is not None:
                    observer(dict(event))
                if len(survivors) == 1:
                    subkey[byte] = next(iter(survivors))
                    break
            else:
                return dict(master_key=None, round_keys=keys, history=history, candidate_counts=counts,
                    used=used, complete=False, blocked_round=number)
        keys[number] = int.from_bytes(subkey, "big")
    master = reverse_sm4([keys[number] for number in range(28, 32)])
    return dict(master_key=master, round_keys=keys, history=history, candidate_counts=counts,
        used=used, complete=True)
```

9.7 Reversing the SM4 key schedule

After recovering four consecutive 32-bit subkeys, the code reverses the key schedule and removes the initial FK constants.

Listing 9.12: Reverse SM4 key expansion, from piret.py

```
def reverse_sm4(final_keys, observer=None):
    """Four consecutive keys rk28..rk31 are REQUIRED, not merely rk31."""
    if len(final_keys) != 4:
        raise ValueError("SM4 reversal requires four consecutive final subkeys.")
    words = [None] * 32 + list(final_keys)
    for index in range(31, -1, -1):
        value = words[index+1] ^ words[index+2] ^ words[index+3] ^ sm4.CK[index]
        words[index] = words[index+4] ^ sm4.linear_key(sm4.sbox(value))
        if observer:
            observer(index, words[index])
    return struct.pack(">4I", *(words[i] ^ sm4.FK[i] for i in range(4)))
```

9.7.1 Independent terminal validation

The controlled-byte model also runs independently in `piret.py`. The following excerpt shows the real output of the supplied code:

Listing 9.13: Recorded SM4 controlled-byte DFA execution excerpt

```

================================================================================
SM4: NORMAL ENCRYPTION
Round 00: A=F002C39E B=18E992B1 L(B)=26D99622 new=27FAD345
Round 01: A=EE1470AC B=C62AD40D L(B)=2820815D new=A18B4CB2
Round 02: A=AA4F1C7D B=23A849C4 L(B)=EF1D58B2 new=11C1E22A
Round 03: A=EC195DAA B=C5445623 L(B)=BA47D0FE new=CC13E2EE
Round 04: A=4A2A2C82 B=3C0BED8A L(B)=DF868890 new=F87C5BD5
Round 05: A=52C45770 B=81BB8BD4 L(B)=92A94BE5 new=33220757
Round 06: A=B1F637DF B=DB4D95B0 L(B)=663520BD new=77F4C297
Round 07: A=98DCAF44 B=ADB86FF3 L(B)=B6851005 new=7A96F2EB
Round 08: A=9B600757 B=301EB78B L(B)=DFA69BAA new=27DAC07F
Round 09: A=9DE0BDBE B=8C896C5B L(B)=71FF084E new=42DD0F19
```

```
Round 10: A=DC966E60 B=B81A781E L(B)=CF511895 new=B8A5DA02
Round 11: A=A3474E33 B=2EBA4FA9 L(B)=EAE7D511 new=907127FA
Round 12: A=0381926D B=FEBF5D21 L(B)=AC4FEBFC new=8B952B83
Round 13: A=939943CC B=A493FC1F L(B)=96F67340 new=D42B7C59
Round 14: A=8B75648F B=CED363A1 L(B)=97598233 new=2FFC5831
Round 15: A=60069A4A B=1E3D323C L(B)=66EF4F72 new=F69E6888
Round 16: A=DC69F8C8 B=B8227911 L(B)=24B11947 new=AF2432C4
Round 17: A=05F35DDE B=E1EC56B4 L(B)=3935B407 new=ED1EC85E
Round 18: A=7823DB74 B=4CF4129F L(B)=7A5FE213 new=55A3BA22
Round 19: A=85BD0481 B=C76CCCBF L(B)=E4D57022 new=124B18AA
Round 20: A=42680EC9 B=A7252CD9 L(B)=C5C3409B new=6AE7725F
Round 21: A=B5C5D18D B=FDDDC161 L(B)=19D569A7 new=F4CBA1F9
Round 22: A=4B725B6C B=19464B01 L(B)=486E4032 new=1DCDFA10
Round 23: A=1A00D498 B=13D6A5AD L(B)=3DBD1EA9 new=2FF60603
Round 24: A=716B85E6 B=003BC777 L(B)=85153D83 new=EFF24FDC
Round 25: A=C0E8A67F B=8D65CA9E L(B)=9B2FCA8C new=6FE46B75
Round 26: A=A1C2A841 B=F6AFC007 L(B)=94F9AABD new=893450AD
Round 27: A=F85A7885 B=790F4CC7 L(B)=5465894F new=7B938F4C
Round 28: A=DFCE82C0 B=B05A8A8D L(B)=BC9C0D9A new=536E4246

... [intermediate observations omitted from this printed excerpt]

Survivors: 54
DFA RECOVERED rk31 = 9124A012 / PASS
DFA RECOVERED rk30 = 01CF72E5 / PASS
DFA RECOVERED rk29 = 62293496 / PASS
DFA RECOVERED rk28 = 428D3654 / PASS
SM4 MASTER KEY: 0123456789ABCDEFFEDCBA9876543210 / PASS
32 faulty outputs. All four subkeys came from DFA, not references.
```

9.8 Interpretation of the results

The terminal output demonstrates successful *software-model* recovery of the AES-128 master key and the SM4 master key. In the SM4 Giraud-inspired known-bit demonstration, the program reports 32 faulty outputs, the four recovered round keys, and a final successful verification. The corresponding values are:

$$rk_{31} = \texttt{9124A012}, \qquad rk_{30} = \texttt{01CF72E5},$$
$$rk_{29} = \texttt{62293496}, \qquad rk_{28} = \texttt{428D3654},$$
$$MK = \texttt{0123456789ABCDEFFEDCBA9876543210}.$$

This extends the earlier pedagogical last-round discussion: the supplied project *does* demonstrate four-subkey recovery under its simulated, selectable-round fault oracle. It does *not* establish that the same fault precision or success rate is achievable through ChipWhisperer glitches on the STM32 target.

References and Scope

1. Student-provided handwritten notes, *AES/SM4 notes*, 27 pages, including AES fundamentals, Piret–Quisquater, SM4 fundamentals, and an SM4 DFA example.
2. NIST, *FIPS 197: Advanced Encryption Standard (AES)* (algorithm and official AES test vector).
3. ISO/IEC 18033-3, *Encryption algorithms — Part 3: Block ciphers* (SM4 specification); SM4 test vector as reproduced in the supplied notes.

Note. The SM4 DFA chapter uses a deliberately strong fault assumption: known XOR magnitude at a specified S-box input. Experimental fault injection must establish whether that assumption is justified. The included Python code demonstrates full recovery with a simulated multi-round fault oracle; no full physical recovery of all four SM4 final round keys is claimed here.